

Documento de Apresentação do Aplicativo

1. Visão geral do aplicativo

Nome do app

Com base no código, há duas referências de nome:

- **Einstein** (metadata em `src/app/layout.tsx` com o título “Einstein - Portal do Aluno”)
- **Einstein Cope** (interface da tela de login e sidebar)

Conclusão: o nome aparece como **Einstein / Einstein Cope** no estado atual do projeto.

Objetivo principal

Oferecer um portal acadêmico para organização da rotina escolar, com foco em:

- tarefas;
- provas;
- calendário de atividades;
- grade de horários;
- arquivos de apoio.

Problema que resolve

O app centraliza informações acadêmicas que normalmente ficam dispersas (mensagens, anotações e avisos). Assim, ajuda estudantes e administradores da turma a visualizar responsabilidades e datas importantes em um único ambiente.

2. Descrição das funcionalidades

Importante: esta seção descreve apenas funcionalidades e é bem técnica.

2.1. Autenticação por e-mail e senha

Onde está no código:

- `src/app/(auth)/login/page.tsx`
- `src/app/(auth)/login/actions.ts`

O que faz:

- Exibe formulário de login (e-mail/senha).
- Executa autenticação via Supabase (`signInWithPassword`).
- Em caso de erro, retorna mensagem ao usuário.

Interação do usuário:

1. Usuário acessa `/login`.
2. Preenche e-mail e senha.
3. Clica em “Entrar”.
4. Se autenticado, é redirecionado conforme o perfil (admin ou aluno).

2.2. Redirecionamento inicial automático

Onde está: `src/app/page.tsx`

O que faz:

- A rota raiz (/) redireciona direto para `/login` .

2.3. Controle de acesso por perfil (admin x aluno)

Onde está:

- `src/app/dashboard/page.tsx`
- `src/app/dashboard/layout.tsx`
- `src/app/dashboard/admin/layout.tsx`
- `src/app/dashboard/student/layout.tsx`

O que faz:

- Lê usuário autenticado.
- Consulta tabela `profiles` para obter `role` .
- Direciona o usuário para:
 - `/dashboard/admin` se for admin;
 - `/dashboard/student` se for aluno (`user`).
- Bloqueia acesso indevido entre áreas.

Interação do usuário:

- Ao entrar no dashboard, o usuário vai automaticamente para a área correta.

2.4. Dashboard do administrador

Onde está: `src/app/dashboard/admin/page.tsx`

O que faz:

- Exibe cartões de navegação para:
 - **Criar Tarefa** (`/dashboard/admin/tasks/new`)
 - **Criar Prova** (`/dashboard/admin/exams/new`)
 - **Gerenciar Arquivos** (`/dashboard/admin/files`)

Observação importante:

- As pastas dessas rotas existem, mas no recorte analisado não há páginas implementadas dentro delas.
- Portanto, o fluxo de criação/edição completo está **não identificado claramente**.

2.5. Dashboard do aluno

Onde está: `src/app/dashboard/student/page.tsx`

O que faz:

- Exibe saudação personalizada com nome do perfil.
- Mostra atalhos para:
 - Calendário
 - Provas
 - Horário
 - Avaliações

Observação:

- As pastas dessas seções existem, porém sem páginas implementadas no recorte analisado.
- Conteúdo completo dessas telas está **não identificado claramente**.

2.6. Navegação lateral responsiva (desktop/mobile)

Onde está:

- `src/components/dashboard/Sidebar.tsx`
- `src/components/dashboard/MobileWrapper.tsx`

O que faz:

- Renderiza sidebar com links específicos por perfil.
- Em mobile, abre/fecha menu lateral em formato drawer.
- Exibe bloco de usuário logado (inicial + nome + papel).
- Permite logout.

Interação do usuário:

- Navega por links do menu.
- Em celular, usa botão de menu (ícone) para abrir a navegação.
- Clica em "Sair" para encerrar sessão.

2.7. Logout

Onde está: `src/components/dashboard/Sidebar.tsx`

O que faz:

- Chama `supabase.auth.signOut()`.
- Redireciona para `/login`.

2.8. Visualização de tarefas em calendário

Onde está: `src/components/dashboard/CalendarView.tsx`

O que faz:

- Exibe calendário mensal com navegação entre meses.
- Permite alternar modo de visualização:
 - `due` (tarefas para entregar)
 - `assigned` (tarefas passadas)
- Permite selecionar um dia para ver lista de tarefas.
- Exibe título e descrição das tarefas.

Interação do usuário:

1. Abre calendário.
2. Alterna entre "Entregas" e "Passadas".
3. Clica em um dia.
4. Visualiza tarefas do dia selecionado.

Observação:

- O checkbox de tarefa existe no componente, mas persistência de conclusão por usuário está **não identificado claramente** no fluxo atual.

2.9. Visualização de provas em calendário

Onde está: `src/components/dashboard/ExamsCalendarView.tsx`

O que faz:

- Exibe calendário mensal com marcação de dias com prova.
- Ao selecionar data, lista provas e conteúdo (`title` e `content`).
- Traz destaque visual para sexta-feira (regra visual do componente).

Interação do usuário:

1. Abre visão de provas.
2. Navega entre meses.
3. Seleciona dia.
4. Lê conteúdos das provas da data.

2.10. Estrutura de API (exemplo)

Onde está: `src/app/api/hello/route.ts`

O que faz:

- Endpoint simples `GET /api/hello` que retorna `{ message: 'Hello, World!' }`.

Observação:

- Não representa lógica acadêmica principal; é uma rota de exemplo.
-

3. Fluxo de uso do usuário

3.1. Primeiro acesso

1. Usuário entra na URL principal `/`.
2. Sistema redireciona para `/login`.
3. Usuário informa e-mail e senha.
4. Sistema autentica no Supabase.

3.2. Direcionamento por perfil

5. Sistema consulta `profiles.role`.
6. Se `admin`, envia para `/dashboard/admin`.
7. Se `user`, envia para `/dashboard/student`.

3.3. Jornada do aluno

8. Aluno acessa visão geral com atalhos.
9. Aluno navega para seções como calendário, provas, horário e avaliações.
10. Em telas com componentes de calendário, seleciona datas para ver detalhes.
11. Pode sair da conta pelo menu lateral.

3.4. Jornada do administrador

8. Admin acessa painel com ações de gestão.
9. Admin navega para criação de tarefa/prova e gerenciamento de arquivos.
10. Pode sair da conta pelo menu lateral.

Limitação de evidência:

- O comportamento completo das telas de criação/manutenção (CRUD) está **não identificado claramente**, pois as páginas dessas rotas não aparecem implementadas no recorte atual.
-

4. Como o sistema funciona (baseado no código)

Importante: esta seção descreve apenas funcionalidades e é bem técnica.

4.1. Tecnologias utilizadas

Frontend

- **Next.js (App Router)**
- **React**
- **TypeScript**
- **Tailwind CSS** (com plugins forms/typography)
- **lucide-react** para ícones

Backend / BaaS

- **Supabase**
 - Autenticação (`auth`)
 - Banco PostgreSQL
 - Políticas RLS

4.2. Estrutura geral da aplicação

- `src/app` : rotas e layouts (login, dashboard, API).
- `src/components/dashboard` : componentes reutilizáveis da interface (sidebar, wrappers e calendários).
- `src/utils/supabase` : clientes Supabase para browser e server.
- `src/types` : modelos de dados (`Task` , `Exam` , `Profile` , etc.).
- `scripts/*.sql` : criação/correções do banco e políticas.

4.3. Lógica de autenticação e autorização

- Login via `signInWithPassword` .
- Após login, consulta à tabela `profiles` para identificar papel.
- Redirecionamento condicional para área de admin ou aluno.
- Layouts de área reforçam bloqueio de acesso por perfil.

4.4. Banco de dados (evidência em SQL)

No script `scripts/setup_database.sql` , há tabelas para:

- `profiles`
- `tasks`
- `exams`
- `schedule_items`
- `file_attachments`

Há também:

- Trigger `handle_new_user` para criar perfil automaticamente no cadastro.
- RLS habilitado.

- Políticas de leitura para autenticados e gestão restrita a admin (com ajustes adicionais em scripts de correção).

4.5. Integrações

- **Integração identificada claramente:** Supabase.
- **Outras integrações externas:** não identificadas claramente no código analisado.

4.6. Estado de maturidade funcional (com base no código)

- Implementado com clareza:
 - login,
 - controle por papel,
 - shell de dashboards,
 - componentes visuais de calendário,
 - navegação responsiva,
 - estrutura de dados e scripts de banco.
- Parcial / não identificado claramente:
 - telas finais de CRUD de tarefas/provas/arquivos,
 - notificações,
 - fluxo operacional completo de avaliação/horário/arquivos na área do aluno,
 - persistência de conclusão de tarefas por usuário.

5. Conclusão executiva

O aplicativo apresenta uma base sólida para um portal acadêmico de turma, com autenticação, separação de perfis, navegação organizada e componentes de visualização de tarefas e provas em calendário. A arquitetura usa Next.js + Supabase de forma coerente e moderna, com políticas de acesso no banco para segurança.

No estado atual do código, o projeto demonstra forte avanço na camada de estrutura e experiência de navegação, enquanto parte das telas operacionais finais ainda depende de implementação adicional para fechar o ciclo completo de uso do representante/aluno/admin.